

Implementando Padrões de Teste (Test Patterns) em um Serviço de Checkout de E-commerce

Arthur Curi Kramberger

Matrícula: 729488

Análise e Construção de Software

Resumo

Este relatório apresenta a implementação de padrões de teste (Test Patterns) aplicados a um serviço de Checkout de E-commerce. São abordados os padrões Object Mother e Data Builder para criação de dados de teste, além dos padrões Stub e Mock para isolamento de dependências externas. O trabalho demonstra como esses padrões previnem Test Smells e contribuem para uma suíte de testes sustentável.

I. INTRODUÇÃO

No desenvolvimento de software, testes automatizados são fundamentais para garantir a qualidade do código. Porém, testes mal escritos podem se tornar um fardo: difíceis de ler, frágeis e custosos de manter. Os Test Smells, como Setup Obscuro e Testes Frágeis, são sintomas de testes que não seguem boas práticas de design.

Este trabalho aplica padrões de teste consagrados para resolver dois problemas centrais: (1) a criação de dados de teste de forma legível e flexível, utilizando Object Mother e Data Builder; e (2) o isolamento de dependências externas, utilizando Stubs e Mocks. O sistema sob teste (SUT) é um CheckoutService que processa pedidos de e-commerce, integrando gateway de pagamento, repositório de dados e serviço de e-mail.

II. PADRÕES DE CRIAÇÃO DE DADOS

A. Por que CarrinhoBuilder em vez de CarrinhoMother?

O padrão Object Mother é ideal para entidades simples e estáveis, como o User, que possui poucos atributos e variações previsíveis (PADRAO ou PREMIUM). Para o User, criamos UserMother com métodos estáticos como umUsuarioPadrao() e umUsuarioPremium().

Já o Carrinho é um objeto complexo: pode ter diferentes usuários, quantidades variáveis de itens, valores distintos, ou estar vazio. Se usássemos um Object Mother para o Carrinho, teríamos uma explosão combinatória de métodos: carrinhoVazio(), carrinhoComUmItem(), carrinhoPremiumComDoisItens(), etc.

O Data Builder resolve isso com uma API fluente que permite compor o objeto passo a passo, tornando explícito apenas o que é relevante para cada teste. Isso elimina o Test Smell de Setup Obscuro.

B. Exemplo: "Antes" vs. "Depois"

Antes (setup manual complexo e obscuro):

```
// Setup manual - difícil de ler
const user = new User(2, 'Maria',
  'premium@email.com', 'PREMIUM');
const item1 = new Item('Camiseta', 100);
const item2 = new Item('Calça', 100);
const carrinho = new Carrinho(
  user, [item1, item2]);
```

Depois (usando Data Builder):

```
// Setup com Builder - legível e claro
const carrinho = new CarrinhoBuilder()
  .comUser(UserMother.umUsuarioPremium())
  .comItens([
    new Item('Camiseta', 100),
    new Item('Calça', 100)
  ])
  .build();
```

C. Como o Builder melhora legibilidade e manutenção

1) Legibilidade: O Builder torna o teste auto-documentado. Ao ler .comUser(umUsuarioPremium()), o leitor entende imediatamente o cenário sem precisar decifrar parâmetros de construtores.

2) Manutenção: Se o construtor de Carrinho mudar (ex: adicionar campo 'cupom'), apenas o Builder precisa ser atualizado. Todos os testes que usam o Builder continuam funcionando sem alteração.

3) Foco: Cada teste explicita apenas os dados relevantes para seu cenário. Valores padrão sensatos no construtor do Builder eliminam ruído.

III. PADRÕES DE TEST DOUBLES

A. Identificação no teste de sucesso Premium

No teste 'quando um cliente Premium finaliza a compra', três dependências externas são substituídas por duplões de teste:

- GatewayPagamento: Stub (retorna {success: true})
- PedidoRepository: Stub (retorna pedido com ID)
- EmailService: Mock (verificamos a chamada)

B. Por que GatewayPagamento é um Stub?

Um Stub é um duplê que fornece respostas pré-definidas para controlar o fluxo de execução do SUT. O GatewayPagamento é configurado para retornar `{success: true}`, permitindo que o teste avance para as etapas seguintes do checkout.

A verificação principal sobre o gateway é de Estado: verificamos que o valor passado ao método `cobrar()` foi R\$180,00 (com desconto aplicado). Não estamos interessados em como o gateway processa internamente, apenas que recebeu o valor correto. Isso caracteriza Verificação de Estado (State Verification).

C. Por que EmailService é um Mock?

Um Mock é um duplê usado para Verificação de Comportamento (Behavior Verification). O EmailService representa um efeito colateral: o envio de e-mail não retorna um valor que afete o fluxo do checkout, mas é uma ação importante que deve ocorrer.

Por isso, verificamos:

- Que `enviarEmail()` foi chamado exatamente 1 vez
- Que foi chamado com o e-mail correto do usuário
- Que a mensagem contém o ID do pedido

Essa é a essência da distinção proposta por Martin Fowler [1]: Stubs controlam entradas indiretas (o que o SUT recebe), enquanto Mocks verificam saídas indiretas (o que o SUT faz com o mundo externo).

D. Resumo comparativo

Dependência	Tipo	Verificação
Gateway	Stub	Estado (valor)
Repository	Stub	Estado (retorno)
EmailService	Mock	Comportamento

O código do teste demonstra claramente essa separação. O `jest.fn()` com `mockResolvedValue` configura o Stub (controle de retorno), enquanto as asserções `toHaveBeenCalled` e `toHaveBeenCalledTimes` verificam o comportamento do Mock.

IV. CONCLUSÃO

O uso deliberado de Padrões de Teste demonstrou ser uma estratégia eficaz para prevenir Test Smells e construir uma suíte de testes sustentável. Os principais benefícios observados foram:

- 1) Prevenção de Setup Obscuro: O Data Builder torna cada teste auto-explicativo, eliminando a necessidade de decifrar construtores complexos com múltiplos parâmetros posicionais.
- 2) Eliminação de Testes Frágeis: Ao isolar dependências externas com Stubs e Mocks, os testes não quebram por mudanças em serviços externos (banco de dados, APIs de pagamento, servidores SMTP).
- 3) Separação de responsabilidades na verificação: Stubs permitem focar na lógica de negócio (o desconto foi calculado corretamente?), enquanto Mocks garantem que efeitos colaterais importantes ocorrem (o e-mail foi enviado?).
- 4) Manutenibilidade: Quando o domínio evolui, apenas os Builders precisam ser atualizados, não dezenas de testes individuais. Isso reduz drasticamente o custo de manutenção da suíte.
- 5) Padrão AAA (Arrange-Act-Assert): Os padrões aplicados naturalmente guiam o desenvolvedor a estruturar testes com separação clara entre preparação, execução e verificação, melhorando a legibilidade.

Em suma, investir em padrões de teste não é overhead — é uma prática de engenharia que transforma testes de um custo contínuo em um ativo de longo prazo, capaz de evoluir junto com o sistema que protege.

REFERÊNCIAS

- [1] M. Fowler, "Mocks Aren't Stubs", martinfowler.com, 2007.
- [2] G. Meszaros, "xUnit Test Patterns: Refactoring Test Code", Addison-Wesley, 2007.
- [3] Jest Documentation, "Mock Functions", jestjs.io/docs/mock-functions.
- [4] R. C. Martin, "Clean Code: A Handbook of Agile Software Craftsmanship", Prentice Hall, 2008.